

"line-edit-pkg", a Readline REPL for Common Lisp Written in Common Lisp

This program features capabilities equivalent to a readline library for Common Lisp, written in Common Lisp. It includes various functionalities such as:

- Emacs/vi/WordMaster compatible commands.
- Searching and executing past history whose prefix matches the input string.
- Completion input of symbol names whose prefix matches the input string.
- Intuitive modification of the REPL (Read Eval Print Loop) prompt.
- Stack management of the package movement order.
- Multi-language support for messages.

Implementing Multiple Types of Editor Modes by Registering Combinations of Key Sequences and Editing Functions

As introduced in detail in each respective section, the functions constituting the editing commands are crafted to have a one-to-one correspondence with Emacs commands. Since it interpolates a mechanism to define the combination of a key sequence for command execution and the function to be executed just like real Emacs, vi and WordMaster compatible commands require little more than preparing their respective key sequences for command execution. If a corresponding capability does not exist in Emacs, the implementation is completed by adding a small function that combines a few predefined editing functions.

History Function Supports Both Readline Style and C-shell Style

The history function is a capability for recalling past inputs to reuse them as they are, or by rewriting them slightly. Although individual key assignments may differ depending on the line-editing function (Emacs mode, vi mode, WordMaster mode), in Emacs mode, typing Control-p (pressing the Control key while typing the p key; hereinafter written as C-p) displays past history in order. Typing C-p after entering the first few characters backtracks through only the past history whose prefix matches the entered characters. Once you reach the target history, simply typing the Return key inputs the displayed line as the input line. The C-shell style history function shown in Table 1 can also be used. The size of the history to be retained is defined at runtime. There is no upper limit, but the initial value is set to 1024 lines. History exceeding the set value is deleted in order starting from the oldest, and the latest 1024 lines become the target of history recall. At the end of a REPL session, history information is automatically saved to a file, and the previous content is loaded at the start of the next REPL session.

Approximately 800 Types of Common Lisp Functions Registered as Completion Candidates

The completion function for symbol names is a capability that, when the Tab key (initial configuration value) is typed after entering a few characters, sequentially presents completion candidates whose input prefix matches, and confirms the selection when the confirmation key (changeable for each editing mode; the initial

value in Emacs mode is C-j) is typed. Candidates can be selected from two types: "short candidates" and "long candidates". In the case of a symbol name, a "short candidate" is only the symbol name, whereas a "long candidate" becomes a symbol name with a package name.

It features a file in which all function names and macro names for approximately 800 types of Common Lisp are registered in advance as completion candidates. In the case of Common Lisp function names and macro names, "short candidates" display only the function name or macro name, while "long candidates" display a string that includes argument and return value information. The "short candidates" and "long candidates" switch each time the Space key is typed. Incidentally, typing the Tab key with nothing entered sequentially displays all approximately 800 types of function names and macro names as candidates. Of course, it can be canceled at any point.

In addition, the displayed completion candidates are shown in priority order along an exponential decay curve based on the number of times the completion candidate was confirmed and the time information at confirmation. Candidates recalled more recently and candidates selected more frequently are lined up at the beginning. The "half-life" given to this exponential decay curve can also be configured freely. Giving a longer time makes the ranking harder to rise, but once risen, the ranking becomes harder to fall. If the configured time is short, the ranking fluctuates sensitively according to recent selections.

REPL Prompt Can Be Defined Intuitively

The REPL prompt can be freely changed with an intuitive definition. You simply specify the following keywords in the order you want them displayed. For example, if you write:

Lisp

```
(history-pkg:set-prompt-element
  "[" ;; Display "[".
  :lisp-type      ;; Implementation name.
  ":"            ;; Display ":".
  #'print-color-string:change-to-green ;; Green from here on.
  :not-cl-user    ;; Displays the package name only when the current package
                  ;; is not [:cl-user].
                  ;; The display color is green.
  #'print-color-string:change-to-blue  ;; Blue again from here on.
  #+sbcl "("      ;; In the case of SBCL, display "(".
  #+sbcl #'print-color-string:change-to-red ;; Red from here on.
  #+sbcl :heap-size ;; In the case of SBCL, display the heap size.
  #+sbcl #'print-color-string:change-to-blue  ;; Blue again from here on.
  #+sbcl ")"      ;; In the case of SBCL, display ")".
  #+(not sbcl) " " ;; In the case of anything other than SBCL, display a space
                  ;; (" ").
  " #"
  :history-number "]"> "    ;; Color and attribute specifications for characters
                          ;; are reset here.
) ;; end history-pkg:set-prompt-element
```

in `init-repl-prompt.lisp`, which registers the initial configuration of the prompt, a REPL prompt of the type shown in the format will be displayed.

```
[sbcl:line-edit-pkg(29.3MB) #1043]> █
```

The above example means that the implementation is SBCL, you are within a package called line-edit-pkg, the heap size is 29.3MB, and the current history number is 1043.

Stack Management of Package Movements

Being able to manage large-scale programs in units of packages is one of the major characteristics of Common Lisp. In the REPL of this program as well, which package the user is currently in can be displayed in the prompt. The package movement history is managed as a stack, being displayed higher up the closer it is to the current stack, and to the right of the package name, an alias is displayed if an alias exists. If the current package is used by other packages, the names of the using packages, the number of external symbols, the number of internal symbols, and the number of inherited symbols of the current package are displayed. These displays can also be hidden through configuration.

```
[sbcl:line-edit-pkg(54.0MB) #1026]> (pushd :support-functions)
2:* #<PACKAGE "SUPPORT-FUNCTIONS"> (debug-print external-command)
1: #<PACKAGE "LINE-EDIT-PKG"> (cl-rl cl-readline)
0: #<PACKAGE "COMMON-LISP-USER"> (cl-user)
パッケージ (history-pkg history-repl line-edit-pkg package-util print-color-string) からユースされています。
support-functionsの外部(61),内部(32),システム由来以外の継承シンボル(0)を取得しました。
[sbcl:support-functions(56.2MB) #1027]> █
```

Movements between packages can perform directory push, pop, rotate down, rotate up, position exchange of the first two package directories, and directory stack display using each function of pushd, popd, rotdd (rotate-down-dir), rotud (rotate-up-dir), exchgd, and dirs. Also, lists of external symbols, internal symbols, inherited symbols, and others can be displayed using the ls function.

Main Messages from the Program Support Multiple Languages

The main messages from the program can be freely switched to messages prepared through advance translation by AI. The default is Japanese messages, but currently, in addition to Japanese (:ja), messages are prepared for:

- English (:en)
- German (:de)
- French (:fr)
- Simplified Chinese (:zh-hans)
- Traditional Chinese (:zh-hant)
- Korean (:ko)
- Zulu (:zu)

Switching is performed by:

Lisp

```
(select-language :en)
```

which switches to English. If these configurations are written inside the system-wide initialization file "line-edit-init.lisp", the settings will be completed at startup. For details, refer to the "line-edit-pkg.lisp Manual".

mail2daigo@gmail.com